# 汇报讲稿（10 分钟）

> 语速提示：正常语速约 160–180 字/分钟，10 分钟约 1600–1800 字。 以下是全文约 1700 字，适合 10 分钟的节奏。括号内为动作/PPT 提示。

---

## 第一部分：问题引入（1.5 分钟）

大家好，我选的题目是 TabularX——一个面向可解释性与可复现性的自动化表格建模工具包。

（翻页）

现在做表格数据的 AutoML，工具已经很多了。MLJAR、AutoGluon、H2O，都能自动帮你选模型、调参数、做预处理。你只需要把数据丢进去，等它跑完，拿一个模型出来。这件事本身已经很成熟了。

但问题是——跑完之后，你拿到的是一个模型。至于数据是怎么预处理的、模型为什么选这个不选那个、每个特征对预测有多大贡献、模型在哪些群体上可能表现不公平——这些信息要么没有，要么散落在代码里，需要你自己去翻。

（翻页）

所以我们的问题很简单：**能不能让 AutoML 的每一次运行，都强制产出一份完整的、可读的实验记录？**

——不是「可选」的导出功能，而是每次 `fit()` 结束之后，一个完整文件夹自动摆在那里，里面是数据概况、划分方案、预处理流水线、模型排名、SHAP 解释图、公平性审计报告，全都有。

这就是 TabularX 要解决的问题。

---

## 第二部分：Baseline——MLJAR 的设计哲学（3 分钟）

在实现之前，我们首先研究了 MLJAR 这个标杆。mljar-supervised 是目前表格 AutoML 中最成熟的开源项目之一，它在设计上有几个值得我们学习的原则。

（翻页）

**第一条：注册表 + 能力声明。**

MLJAR 里每个模型族在注册时，会声明自己「需要哪些预处理」。比如线性模型和神经网络要求标准化，随机森林和 XGBoost 不需要，CatBoost 连类别编码都不需要——因为它自己能处理。框架根据模型的能力声明，自动匹配合适的预处理策略，不做一刀切。

（翻页）

**第二条：Profile 只报事实，不做决策。**

MLJAR 把数据观测和预处理决策分成两层。第一步，`DataInfo` 模块只去「看」数据——哪些列是数值、哪些是类别、哪些有缺失、哪些偏态严重。第二步，`PreprocessingTuner` 再根据「数据事实」和「模型能力」的交集做出决策。Profile 只管描述，不管怎么处理。

（翻页）

**第三条：自动化的实验记录。**

MLJAR 并不是等用户去查日志，而是在每次运行中自动写入结构化的实验记录——顶层有 `data_info.json` 描述列信息、`params.json` 记录运行参数、`leaderboard.csv` 列出所有模型分数；每个模型目录里还有 `framework.json` 记录预处理策略、验证策略、模型参数和训练时间。一个目录打开，这次实验的数据概况、参数配置、模型对比，全都有据可查。

这个理念给了我们一个启发：**实验记录不应该靠用户手动整理，而应该成为框架的默认产出。**

（此外，MLJAR 在工程实现上还有一个值得注意的细节——始终坚持先划分、后预处理，确保统计量严格从训练集计算。这一点在工程上执行得很到位。）

（翻页）

**这三条设计原则给了我们很好的起点。在实现 TabularX 时，我们继承并进一步推进了这些理念。**

---

# 第三部分：TabularX 的设计（3.5 分钟）

## 架构层面（~1.5 分钟）

TabularX 在架构上做了几件事。

（翻页）

**第一，纯函数分层。**

这是 TabularX 最核心的架构决策。我们把整个流水线拆成独立的阶段——Profile、Split、Preprocess、ModelSearch、SHAP、Fairness——每个阶段都是纯函数，只做计算，不做 IO。Profile 只返回 `DataProfile` 对象，不写文件；Split 只返回 `SplitResult`；Preprocess 只返回 `PreprocessedData`。IO——写 HTML、存模型、保存图片——全部交给最外层的 `estimator` 管。

这样做的好处是，每个阶段的测试只需要「给输入、断言输出」，不需要 mock 文件系统。

（翻页）

**第二，Schema 通信协议。**

阶段之间不直接依赖对方的实现，只通过 dataclass 通信。Profile 产出 `DataProfile`，Split 产出 `SplitResult`，Preprocess 产出 `PreprocessedData`——每个 dataclass 就是一份数据契约。只要契约不变，内部实现可以随便换。

值得一提的是，MLJAR 也有类似的概念，但它是把中间数据写到 JSON 文件里落盘；TabularX 用 dataclass 在内存中做类型安全的通信，最后再统一渲染为报告。

在这个框架下，异常管理也变得很自然——所有领域异常继承自 `TabularXError`，用户可以在调用链路上统一 catch。而参数越界这类底层问题直接用 Python 内置的 `ValueError`，不额外包装——因为调用方不需要 catch 一个 `ConfigError` 来知道参数传错了。

（翻页）

**第三，封装设计——Estimator 编排层。**

纯函数层之上，`AutoEstimator` 负责把各个阶段串起来。它封装了完整的调用逻辑：先调 Profile 做数据观测，然后把结果传给 Split，再传给 Preprocess，依次往下。所有 IO——写 HTML、存模型、保存图片——也由它统一管理。

所以用户的使用体验只有两行代码：

```
ae = AutoEstimator(task="classification", output_dir="./output")
ae.fit(X, y, sensitive_features=["gender"])
```

`fit()` 内部自动完成整条流水线，外部只需要关注输入和输出。

（翻页）

**第四，Jinja2 模板渲染。**

每个阶段的数据都落入对应的 dataclass，最后渲染层把这些数据填入 Jinja2 模板，生成独立的 HTML 文件。模板和逻辑彻底分离——如果你不满足于默认的报告风格，可以自己去改模板，不需要改 Python 代码。

## 能力层面（~2 分钟）

架构之上，TabularX 的能力设计沿着数据流自然展开。

（翻页）

**第一，注册表与模型能力声明。**

我们从 MLJAR 继承了注册表机制。每个模型族在注册时，声明自己的任务类型（回归还是分类）、搜索空间、默认参数、以及支持的能力——是否需要标准化、是否需要类别编码、是否支持 feature importance、是否支持 predict_proba。注册表按 binary / multiclass / regression 三个桶管理，框架根据数据自动确定任务类型，再从对应桶里取出可用模型。

（翻页）

**第二，Profile 观测与预处理自动路由。**

这是从 MLJAR 继承的另一条核心理念——Profile 只报事实，不做决策。

第一步，框架去「看」数据——哪些列是数值、哪些是类别、哪些有缺失、哪些偏态严重、哪些需要标准化。这一步与模型无关，只输出一份数据观测报告。

第二步，框架根据 Profile 的观测结果，结合每个模型的能力声明，自动求交集做出预处理决策。线性模型需要标准化，随机森林不需要，CatBoost 连类别编码都不需要——框架自动识别，不做一刀切。

（翻页）

**第三，Optuna 多进程调参。**

模型搜索阶段，每个模型都有自己的声明式搜索空间——用 `ParamDef` 列表描述参数的名字、类型、范围、尺度。调参器统一遍历所有 `ParamDef`，生成 Optuna 的 suggest 调用。搜索空间定义和调参逻辑彻底解耦。

多进程跑下来，每个模型都有独立的调参轨迹和最优参数。而且——这一点我们和 MLJAR 不同——每个成功跑完的模型都有一份自己的独立报告，包含评估指标、混淆矩阵、ROC 曲线、调参轨迹图。对比优势一目了然。

（翻页）

**第四，SHAP 解释与公平性审计。**

最优模型确定之后，框架自动对最优模型做两件事。

一是 SHAP 可解释性分析——生成特征重要性排序、SHAP summary 图、Waterfall 图，告诉你每个特征对预测有多大贡献、正负方向如何。

二是公平性审计——用户只需在 `fit()` 时多传一个参数 `sensitive_features`，指定敏感特征。框架自动把测试集按敏感特征分组，计算每组的 Demographic Parity 和 Equalized Odds 差异，超阈值自动标红告警。不需要额外写代码。

（翻页）

**第五，报告生成——协议数据到最终产出。**

最后一步，所有阶段积累的数据——Profile 的数据观测、Split 的划分方案、Preprocess 的预处理流水线、ModelSearch 的模型排名和每个模型的详细结果、SHAP 的解释图、公平性的审计报告——全部通过 Jinja2 模板渲染，自动生成一系列 HTML 文件。每个 HTML 直接用浏览器打开就能看，一次 `fit()` 跑完，一个完整的实验目录自动成型。